

Calentasker

Fejlesztési dokumentáció



A vizsgaremeket készítették:

Fesető Imre 13.I

Kapus Kevin Kamill 13.D



Tartalom

1	Bevezetés	5
1.1	Feladatcím.....	5
1.2	Rövid ismertető	5
2	Elvárások.....	6
2.1	Operációs rendszer, környezet	6
2.2	Használandó programozási nyelvek	6
2.3	Funkciók	6
3	Szoftver specifikáció.....	8
3.1	Megjelenés	8
4	A Calentasker célja	9
4.1	A rendszer fő funkciói.....	9
5	A Calentasker működésének leírása	10
5.1	Feladat hozzáadása	11
5.2	Feladat módosítása.....	12
5.3	Feladat törlése.....	13
5.4	Feladatok rendezése.....	13
5.5	Feladatok szűrése	14
5.6	Értesítések kezelése	15
6	Szerkezet.....	16
7	Táblák	17
7.1	Felhasználó	17
7.2	Csoport	18
7.3	Csoporttagok	18
7.4	Feladat	19
7.5	Hozzárendelt	20
7.6	Csatolások.....	20
7.7	Hozzászólás	21
8	Tesztkörnyezet és Eszközök.....	22



8.1	Frontend	22
8.2	Backend	22
Frontend		23
9	Összesített mérési eredmények.....	24
10	Tesztesetek	26
10.1	Autentikáció és Felhasználói Folyamatok.....	26
10.1.1	Sikeres bejelentkezés.....	26
10.1.2	Bejelentkezés hibás adatokkal	27
10.1.3	Védett útvonalak biztonsága	28
10.1.4	Kijelentkezés.....	28
10.2	Csoportok és Együttműködés	29
10.2.1	Új csoport felvitele	29
10.2.2	Felhasználó kereső.....	30
10.3	Feladatkezelés	31
10.3.1	Feladat rögzítése	31
10.3.2	Feladat részletei	32
10.3.3	Nézetváltás	33
10.3.4	Feladat eltávolítása.....	34
10.4	Felhasználói Felület és Beállítások.....	35
10.4.1	Felhasználói adatok módosítása	35
10.4.2	Nyelvváltás.....	36
11	Melléklet: Forráskód kivonat	37
11.1	Resources.resource (Közös változók és kulcsszavak):.....	37
Backend		38
12	Tesztesetek	39
12.1	Autentikáció és Felhasználói Folyamatok.....	39
12.1.1	Felhasználó regisztrációja	39
12.1.2	Fiók aktiválása	40
12.1.3	Felhasználó bejelentkezése	41
12.1.4	Fiók törlése	42



12.2	Csoportok és Együttműködés	43
12.2.1	Csoportok létrehozása.....	43
12.2.2	Csoporttag hozzáadása.....	44
12.2.3	Csoport törlése	45
12.3	Feladatkezelés	46
12.3.1	Személyes feladat hozzáadása.....	46
12.3.2	Csoportos feladat létrehozása.....	47
12.3.3	Feladat módosítása	48
12.3.4	Feladatok szűrése	49
12.3.5	Feladat törlése	50
12.4	Lekérések	51
12.4.1	Felhasználók lekérése	51
12.4.2	Adott felhasználó lekérése	51
12.4.3	Csoportok lekérése	52
12.4.4	Adott csoport lekérése	52
12.4.5	Feladatok lekérése	53
12.4.6	Adott feladat lekérése	53



1 Bevezetés

1.1 Feladatcím

Calentasker

1.2 Rövid ismertető

A Calentasker célja, hogy modern és felhasználóbarát megoldást kínáljon a feladatok tervezésére, rendszerezésére és kiosztására. A rendszer lehetőséget biztosít csoportok és alcsoportok létrehozására, ahol a csoportvezető könnyedén kioszthatja a feladatokat a tagok között, határidővel és prioritással ellátva azokat. Az online felületnek köszönhetően a felhasználók bárhol és bármikor hozzáférhetnek feladataikhoz, értesítést kaphatnak a változásokról, és visszajelzést adhatnak a feladatok állapotáról. A rendszer támogatja a feladatok státuszának nyomon követését (például: „új”, „folyamatban”, „kész”), így a csapat minden tagja mindig naprakész információhoz jut. A Calentasker fő célcsoportja minden olyan közösség, amely közösen dolgozik projekteken vagy tevékenységeken - legyen szó iskolai munkáról, munkahelyi csapatmunkáról vagy akár hobbiközösségek szervezéséről. A program előnye az átlátható feladatkezelés, a hatékony kommunikáció támogatása, az időmegtakarítás és a félreértések minimalizálása.



2 Elvárások

2.1 Operációs rendszer, környezet

Bármilyen operációs rendszeren működő böngésző.

2.2 Használandó programozási nyelvek

- Vue.js - Vue router, Axios
- mySQL - Django + Python
- HTML
- CSS
- JavaScript
- Verziókezelő rendszer: GitHub

2.3 Funkciók

A szoftvernek minél több funkciót szeretnénk adni, köztük a legfontosabbak:

Bejelentkezés/regisztráció - A regisztrációhoz meg kell adni a felhasználó: fióknevét, megjelenési nevét (Ha meg akarja változtatni a fiók nevet akkor ez jelenik meg), e-mail címét, jelszavát. Jelszavak erősítése érdekében követelmény lesz a nagy- és kisbetű, illetve legalább egy szám vagy speciális karakter.

- Feladat rögzítése - Minden feladatnak kötelezően meg kell adni címet, illetve ha csoportban lett létrehozva - személyt akinek a feladatot szánja a létrehozó.
- Feladat időzítése - Minden feladatot lehet időzíteni, hogy mettől meddig szánja megcsinálni.
- Feladat időtartamának megszabása - Minden feladatnak meg lehet adni egy időtartamot, melynek lejáratát után a szoftver automatikusan törli ha késznek van megjelölve, illetve átrakja „elmaradt” feladatnak ha nincs késznek megjelölve. Feladatok megosztása - Feladatokat meg lehet osztani más csoportokkal, illetve külön személyekkel.
- Csoportok létrehozása - Csoportokon belül létrehozhatók al-csoportok is. Minden csoportnak kell adni egy nevet, illetve legalább 2 tag kell egy létrehozásához.



- Tagok felvétele -Tagokat alapértelmezetten olvasóként lehet csak felvenni, de ezt a csoportvezető átállíthatja magasabb szintekre is (Olvasó, Kezelő, Moderátor). Csoportvezetés helye egy másik tagnak bármikor átadható.
- Értesítés közelgő feladat kezdeti idejéről, időpontjáról, határidejéről - Ha meg van nyitva akkor szolgáltatáson belül megjeleníti azon feladatokat, melyeknek határideje, kezdete közeleg.
- Feladatok személyhez rendeltetése csoporton belül - Csoporton belül feladat létrehozásánál ki kell adni bizonyos személynek a feladatot, azonban ez változtatható bármikor ha a feladatot a csoportvezető más személynek szeretné átadni.



3 Szoftver specifikáció

A szoftverbe beírt feladatok adatai alapján küldjön értesítést feladatok közelgő kezdete, határideje illetve folyamatban lévő feladatokról, illetve jelenítse meg őket egy könnyen használható, rendezett felületen.

3.1 Megjelenés

A szoftver indítása/weboldal megnyitása után a „kezdőképernyő” egy bejelentkezési-, illetve regisztrációs űrlap, melyben a felhasználó bejelentkezhet/regisztrálhat. Az oldal minden része reszponzív.



4 A Calentasker célja

A Calentasker célja, hogy a felhasználó kényelmesen tudja rendezni, észben tartani feladatait.

A program segítségével a felhasználó:

- Létrehozhat új feladatokat,
- Törölhet, illetve módosíthat létező feladatokat,
- Azon feladatokat megtekintheti naptár-, illetve kártyastílusban,
- Értesítéseket kap a feladatok közelgő határidejéről, kezdetéről

A Calentasker webes felületen működik, tehát böngészőből érhető el, HTML, CSS és JavaScript technológiák segítségével.

4.1 A rendszer fő funkciói

Feladatok kezelése: új feladat hozzáadása, szerkesztése, törlése

Rendezés és szűrés: a feladatok megjelenítése szűrési szempontok alapján (személynek osztott, készenlét, stb.)

Értesítések: figyelmeztetés közelgő határidő vagy kezdet esetén

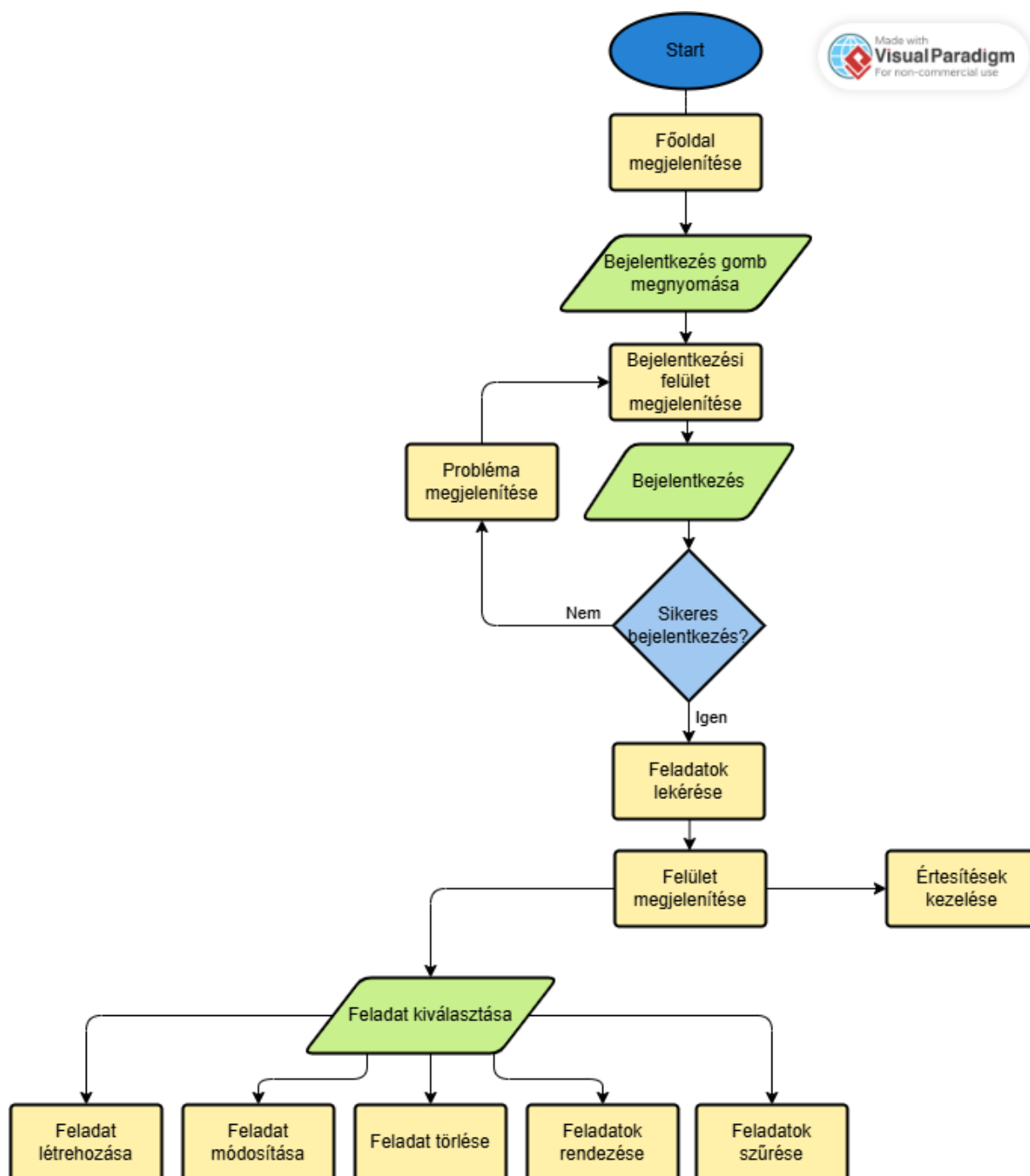
Státuszkezelés: a feladatok állapotának nyomon követése (folyamatban, kész, nincs kész, stb.)

Adatmentés: a feladatok tárolása egy adatbázis segítségével



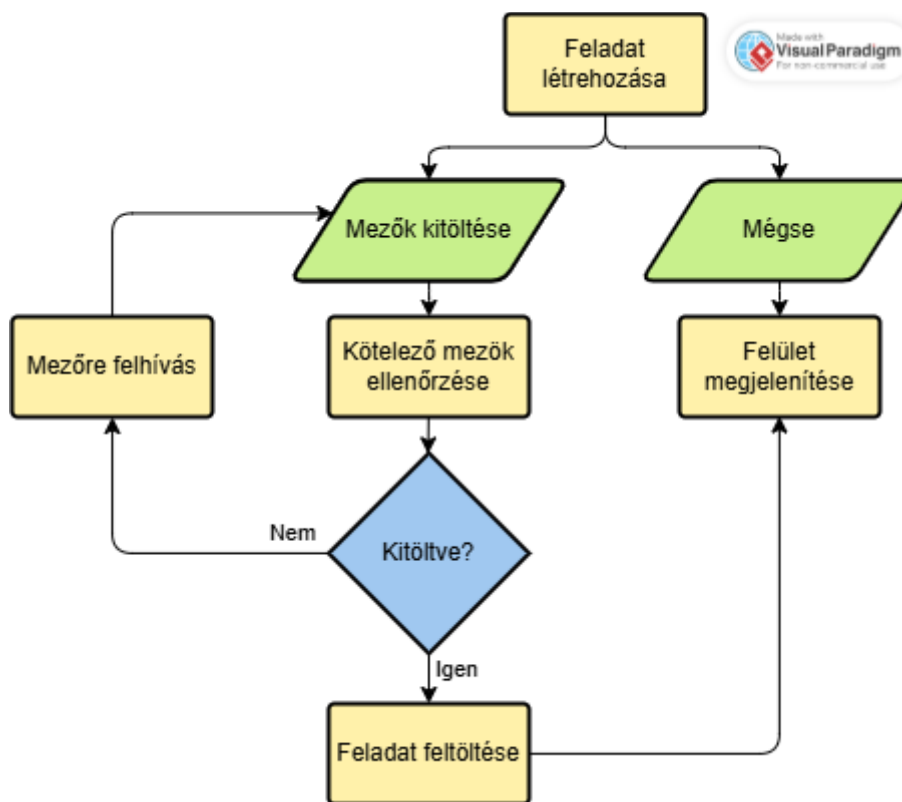
5 A Calentasker működésének leírása

A Calentasker elindításakor a program megjeleníti az alap felületét, amelyen a felhasználó bejelentkezhets. Sikeres bejelentkezés után a program megjeleníti a felhasználó feladatait.





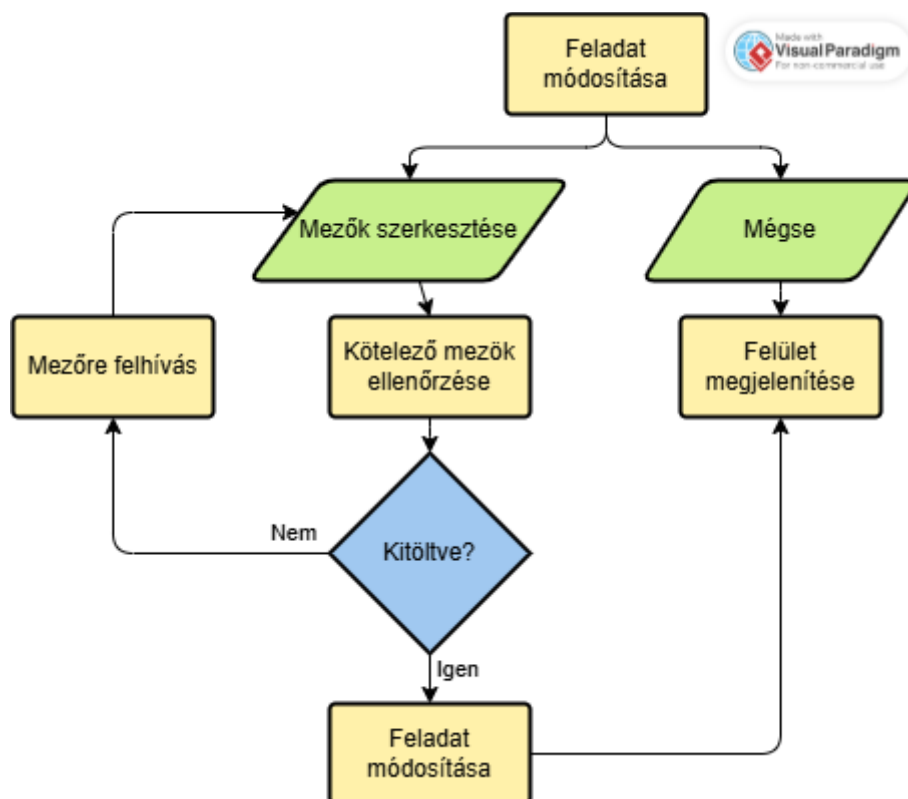
5.1 Feladat hozzáadása



- A felhasználó kitölti a feladat létrehozásához szükséges űrlapot
- A „Hozzáadás” gomb megnyomásakor a program ellenőrzi, hogy minden mező ki van töltve.
- Az adatokból létrejön egy új feladat objektum
- A feladat adatai feltöltődnek az adatbázisba
- Az új feladat hozzáadódik a feladatlistához, és megjelenik a felületen.



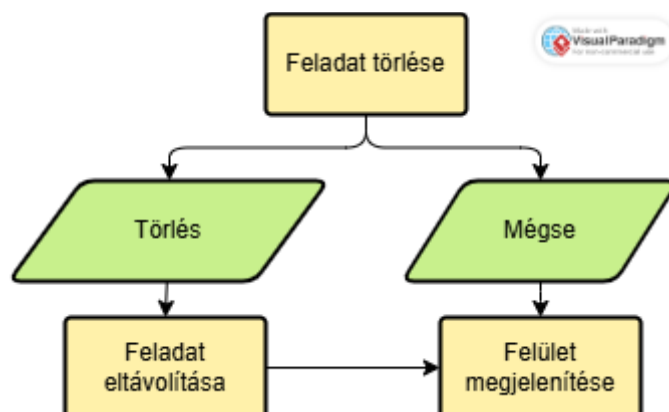
5.2 Feladat módosítása



- A feladat mellett megjelenő „Szerkesztés” gomb megnyomásával a program felismeri, melyik feladatot kell módosítani.
- A „Kész” gomb megnyomásával felhasználó frissítheti az adatokat, majd mentheti a változtatást.

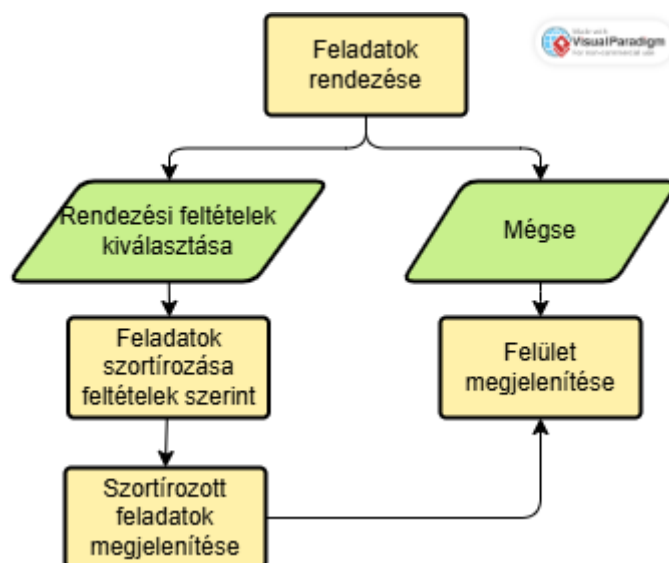


5.3 Feladat törlése



- A feladat mellett megjelenő „Törlés” gomb megnyomásával a program felismeri, melyik feladatot kell módosítani.
- A feladatot a program megkeresi, majd törli az adatbázisból

5.4 Feladatok rendezése



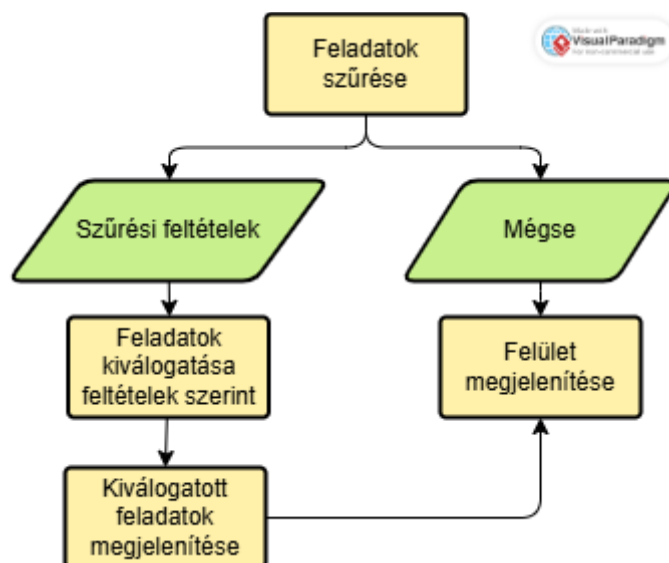
A felhasználó rendezheti a listát:

- Határidő szerint
- Sürgősség szerint

A program a rendezést JavaScript-ben végzi, a lista elemeinek összehasonlításával.



5.5 Feladatok szűrése



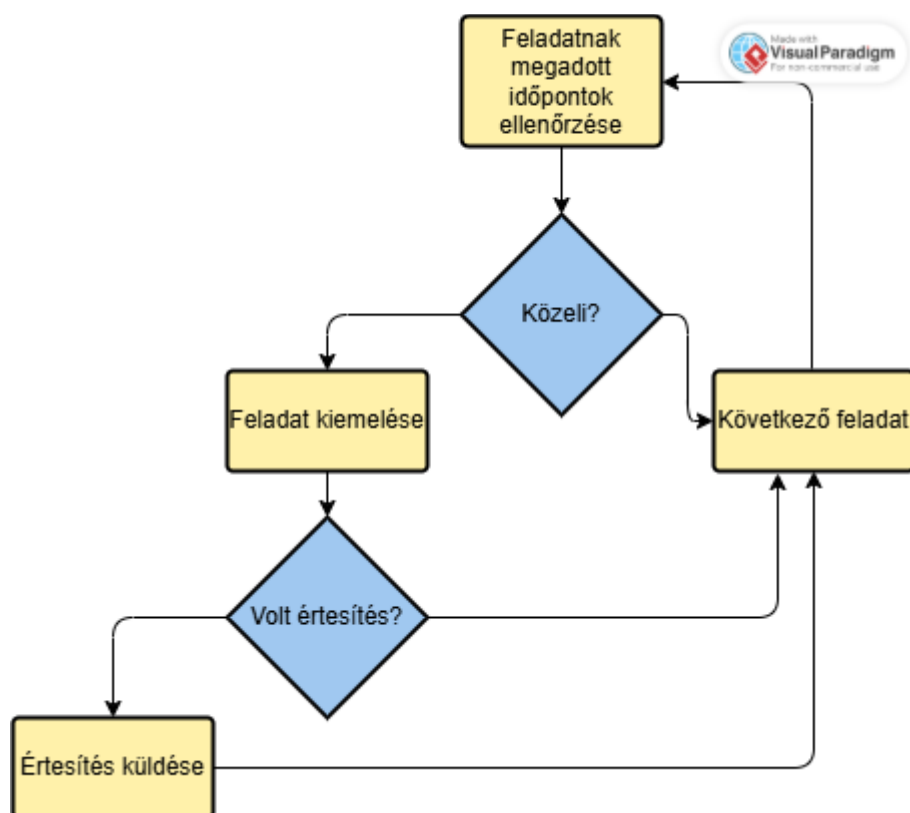
A felhasználó szűrheti a feladatokat:

- készenlét alapján (pl. csak „folyamatban” lévők),
- hozzárendelt személy alapján (pl. csak ehhez a személyhez hozzárendeltek).

A JavaScript a teljes feladatlistából csak azokat jeleníti meg, amelyek megfelelnek a kiválasztott feltételnek.



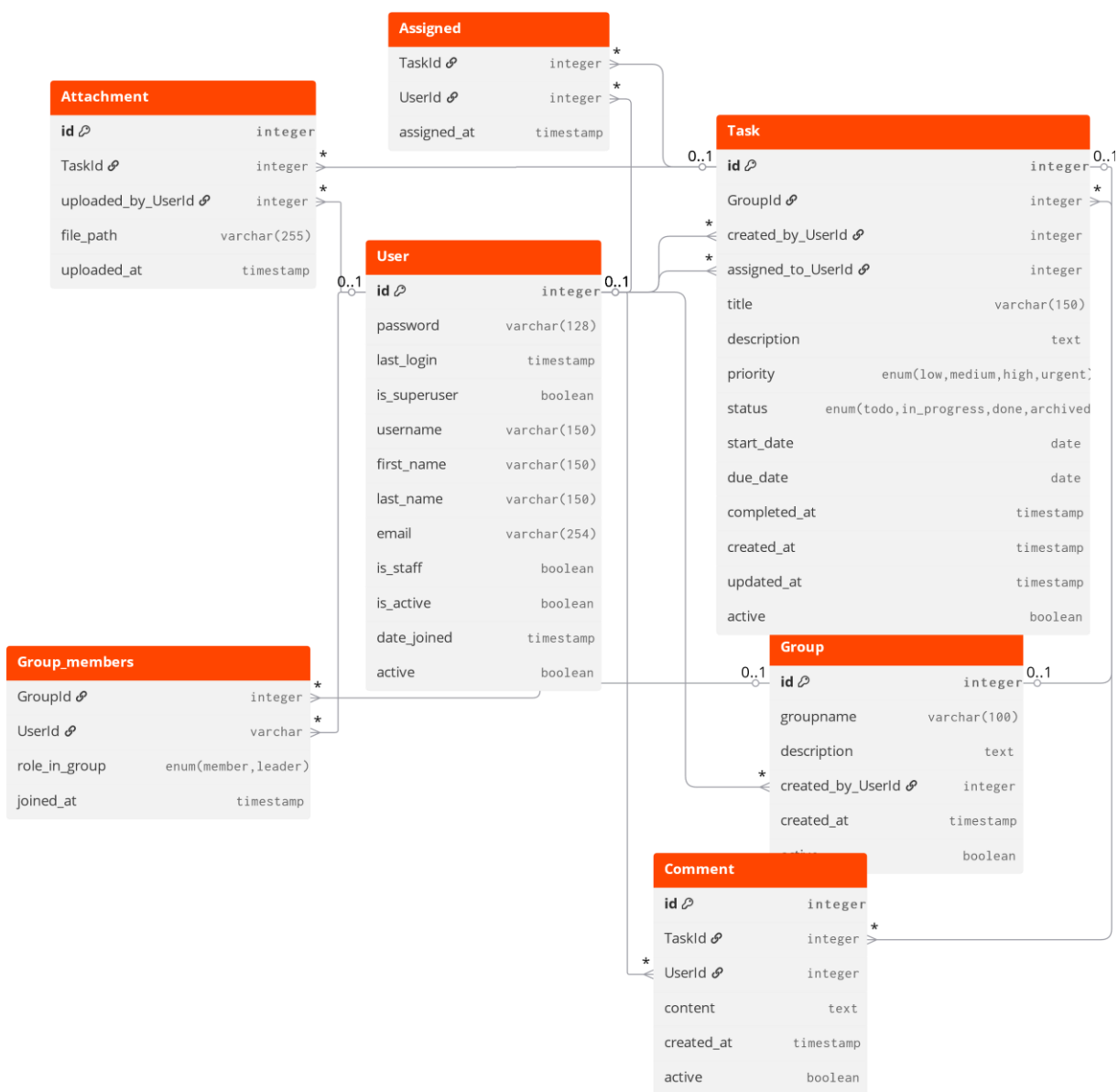
5.6 Értesítések kezelése



A program 12 óránként ellenőrzi a feladatok határidejét. Ha egy feladat határideje 24 órán belül lejár, és még nincs „kész” státuszban, a program külön megjeleníti a feladatot, illetve értesítést küld a feladat közeli határidejéről. A feladat kezdeti idejénél ugyan ezzel az algoritmussal küld értesítést, illetve megjeleníti külön részen, a hamarosan lejárt feladatoktól külön szedve. A cél, hogy a felhasználó ne felejtse el a sürgős teendőket.



6 Szerkezet





7 Táblák

7.1 Felhasználó

Tábla neve

User

Adatok

- id – A felhasználó azonosítója
- password – A felhasználó jelszava
- last_login – A felhasználó utolsó bejelentkezésének ideje
- is_superuser – A felhasználó superuser-e
- username – A felhasználó neve
- first_name – A felhasználó keresztnéve
- last_name – A felhasználó utónéve
- email – a felhasználó email címe
- is_staff – A felhasználó a személyzetbe tartozik-e
- is_active – A felhasználó aktív-e
- date_joined – A felhasználó regisztrálásának ideje
- active – A felhasználói fiók aktív-e

Kapcsolatok

- User.id – Task.assigned_to_UserId
- User.id – Task.created_by_UserId
- User.id – Group.created_by_UserId
- User.id – Comment.UserId
- User.id – Group_Members.UserId



7.2 Csoport

Tábla neve

Group

Adatok

- id – A csoport azonosítója
- groupname – A csoport neve
- description – A csoport leírása (nem kötelező)
- created_by_UserId – A csoportot létrehozó személy azonosítója
- created_at – A csoport létrehozásának ideje
- active – A csoport törölt-e (Változhat, lehet nem lesz itt)

Kapcsolatok

- Group.created_by_UserId – User.id
- Group.id – Group_members.GroupId

7.3 Csoporttagok

Tábla neve

Group_members

Adatok

- GroupId
- UserId
- role_in_group
- joined_at

Kapcsolatok

- Group_members.GroupId – Group.id
- Group_members.UserId – User.id

A Group_members tábla egy kapcsolat a User és a Group tábla között. Ez a tábla segítségével határozható meg ki van melyik csoportban.



7.4 Feladat

Tábla neve

Task

Adatok

- id – A feladat azonosítója
- GroupId – A csoport azonosítója, melyhez a feladat tartozik
- created_by_UserId – A feladatot létrehozó személy azonosítója
- assigned_to_UserId – A feladathoz hozzárendelt személy azonosítója
- title – A feladat címe/neve
- description – A feladat leírása
- priority – A feladat sürgőssége
- status – A feladat állapota
- start_date – A feladat kezdetének ideje
- due_date – A feladat határideje
- completed_at – A feladat befejezésének ideje*
- created_at – A feladat létrehozásának ideje
- updated_at – A feladat frissítésének ideje, ha címét, leírását vagy sürgősségét szerkeszteni kell
- active – A feladat törölve lett-e

**A feladat befejezésének ideje akkor lesz meghatározva, amikor a feladat állapota „kész”-re lesz változtatva. Ha a feladat készenléte visszavonásra kerül bármi ok miatt, ez a mező újra üres lesz.*

Kapcsolatok

- Task.id – Assigned.TaskId
- Task.id – Attachment.TaskId
- Task.id – Comment.TaskId
- Task.GroupId – Group.id
- Task.created_by_UserId – User.id
- Task.assigned_to_UserId – User.id



7.5 Hozzárendelt

Tábla neve

Assigned

Adatok

- TaskId – A feladat azonosítója
- UserId – A felhasználó azonosítója
- assigned_at – A felhasználó feladathoz rendelésének ideje

Kapcsolatok

- Assigned.TaskId – Task.id
- Assigned.UserId – User.id

Az Assigned tábla egy kapcsolat a User és a Task táblák között. Ennek segítségével könnyebben határozzuk meg a feladathoz hozzárendelt személyt.

7.6 Csatolások

Tábla neve

Attachment

Adatok

- id – A csatolás azonosítója
- TaskId – A feladat azonosítója
- uploaded_by_UserId – A csatolt fájlt feltöltött személy azonosítója
- file_path – A fájl helye
- uploaded_at – A fájl feltöltésének ideje

Kapcsolatok

- Attachment.TaskId – Task.id
- Attachment.uploaded_by_UserId – User.id



7.7 Hozzászólás

Tábla neve

Comment

Adatok

- id – A hozzászólás azonosítója
- TaskId – A feladat azonosítója
- UserId – A hozzászóló felhasználó azonosítója
- content – A hozzászólás tartalma
- created_at – A hozzászólás létrehozási ideje
- active – A hozzászólás törölve lett-e

Kapcsolatok

- Comment.TaskId – Task.id
- Comment.UserId – User.id



8 Tesztkörnyezet és Eszközök

8.1 Frontend

A frontend automatizált tesztelése a felhasználói élmény és a komponensek helyes működésének ellenőrzésére irányul.

- **Teszt keretrendszer:** Robot Framework
- **Könyvtár:** SeleniumLibrary
- **Böngésző:** Firefox
- **Módszertan:** Kulcsszó-vezérelt tesztelés (Keyword-Driven Testing), különálló „Resources.resource” fájljal a technikai logika szétválasztásához.

8.2 Backend

A backend automatizált tesztelése az API végpontok, az adatbázis-műveletek, az üzleti logika és a jogosultságkezelés (autentikáció/autorizáció) helyes működésének ellenőrzésére irányul.

- **Teszt keretrendszer:** Django Test Framework (a Python beépített unittest moduljára építve)
- **Könyvtár:** Django REST Framework (`APITestCase` és beépített tesztkliens a HTTP kérések szimulálására)
- **Környezet:** Lokális tesztkörnyezet elszigetelt teszt-adatbázissal (kiegészítve a Postman/Newman eszközeivel a végpontok API-szintű kollekció-teszteléséhez)
- **Módszertan:** API-szintű integrációs és egységtesztelés (Unit & Integration Testing). Objektumorientált megközelítés (Object-Oriented Testing), ahol az ismétlődő technikai lépéseket és a tesztadatok létrehozását különálló, újrafelhasználható `setUp` metódusok végzik a tesztek izolációja és átláthatósága érdekében.



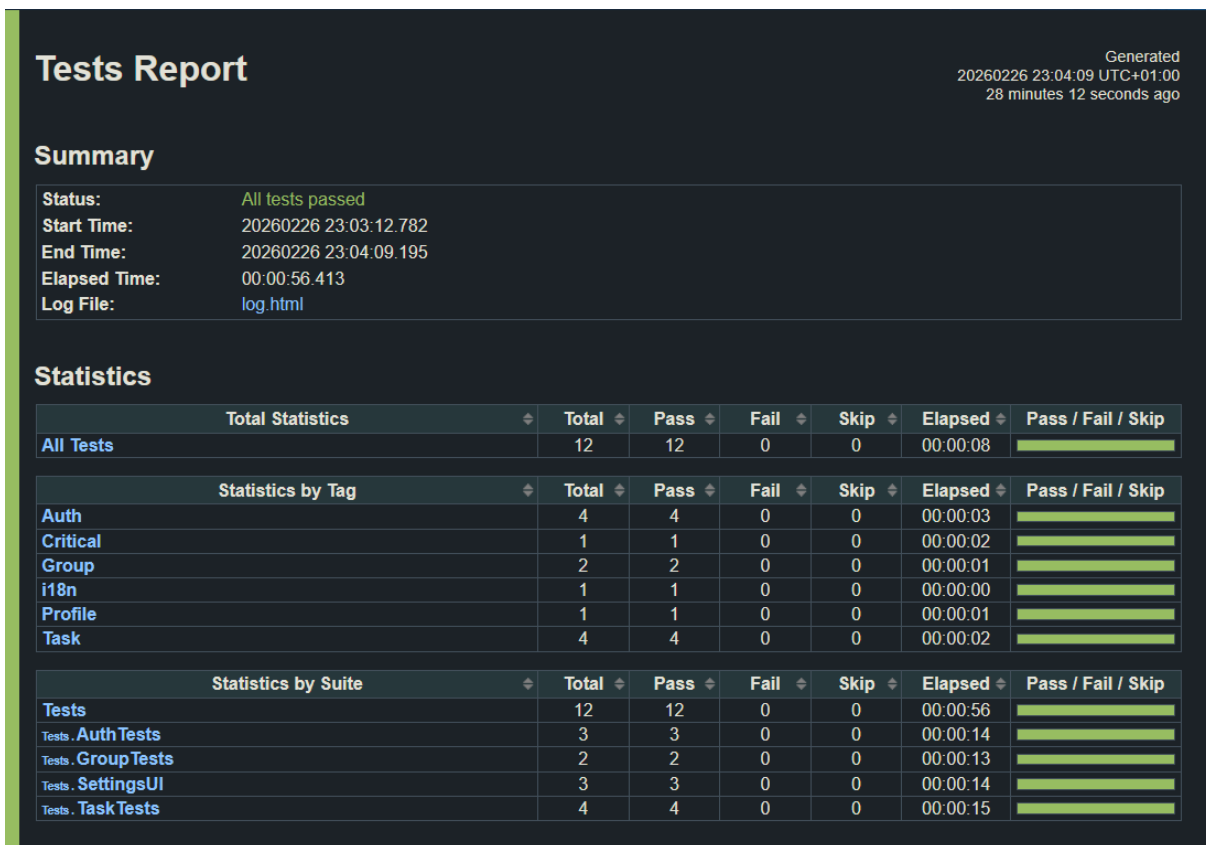
Frontend

Teszt dokumentáció





9 Összesített mérési eredmények



Az összesített riport alapján mind a 12 tesztet sikeresen lefutott.



Tests Log

Generated: 20260226 23:04:09 UTC+01:00
40 minutes 26 seconds ago

Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	12	12	0	0	00:00:08	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Auth	4	4	0	0	00:00:03	
Critical	1	1	0	0	00:00:02	
Group	2	2	0	0	00:00:01	
ifBn	1	1	0	0	00:00:00	
Profile	1	1	0	0	00:00:01	
Task	4	4	0	0	00:00:02	

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Tests	12	12	0	0	00:00:56	
Tests - Auth Tests	3	3	0	0	00:00:14	
Tests - Group Tests	2	2	0	0	00:00:13	
Tests - SettingsUI	3	3	0	0	00:00:14	
Tests - Task Tests	4	4	0	0	00:00:15	

Execution Log

SUITE Tests

Full Name: Tests
Source: C:\Users\Admin\Documents\GitHub\13_S1_8_vizsgaretek\Ends\Frontend\Tests
Start / End / Elapsed: 20260226 23:03:12.782 / 20260226 23:04:09.195 / 00:00:56.413
Status: 12 tests total, 12 passed, 0 failed, 0 skipped

00:00:56.413

REPORT

SUITE AuthTests

Full Name: Tests.AuthTests
Source: C:\Users\Admin\Documents\GitHub\13_S1_8_vizsgaretek\Ends\Frontend\Tests\AuthTests
Start / End / Elapsed: 20260226 23:03:12.782 / 20260226 23:04:09.195 / 00:00:14.340
Status: 3 tests total, 3 passed, 0 failed, 0 skipped

00:00:14.340

GroupTests

Full Name: Tests.GroupTests
Source: C:\Users\Admin\Documents\GitHub\13_S1_8_vizsgaretek\Ends\Frontend\Tests\GroupTests
Start / End / Elapsed: 20260226 23:03:12.782 / 20260226 23:04:09.195 / 00:00:13.358
Status: 2 tests total, 2 passed, 0 failed, 0 skipped

00:00:13.358

SettingsUI

Full Name: Tests.SettingsUI
Source: C:\Users\Admin\Documents\GitHub\13_S1_8_vizsgaretek\Ends\Frontend\Tests\SettingsUI
Start / End / Elapsed: 20260226 23:03:12.782 / 20260226 23:04:09.195 / 00:00:13.640
Status: 3 tests total, 3 passed, 0 failed, 0 skipped

00:00:13.640

SUITE TaskTests

Full Name: Tests.TaskTests
Documentation: Suite for Task Management
Source: C:\Users\Admin\Documents\GitHub\13_S1_8_vizsgaretek\Ends\Frontend\Tests\TaskTests.robot
Start / End / Elapsed: 20260226 23:03:54.168 / 20260226 23:04:09.194 / 00:00:15.026
Status: 4 tests total, 4 passed, 0 failed, 0 skipped

00:00:15.026

SETUP suite.Run Keywords Open Browser To Calorifasker AND Login To System

00:00:09.591

TEARDOWN SeleniumLibrary.Close Browser

00:00:03.030

TEST Create A New Task

00:00:00.688

TEST Task Detail View

Full Name: Tests.TaskTests.Task Detail View
Documentation: Test 5. Click a task and verify detail modal.
Tags: Task
Start / End / Elapsed: 20260226 23:04:04.451 / 20260226 23:04:04.960 / 00:00:00.509
Status: PASS

00:00:00.509

KEYWORD SeleniumLibrary.Go To \${URL}/tasks

00:00:00.207

KEYWORD SeleniumLibrary.Wait Until Element Is Visible css: fadeInItem timeout=5s

00:00:00.053

KEYWORD SeleniumLibrary.Wait And Click css: fadeInItem_viewBtn

00:00:00.168

KEYWORD SeleniumLibrary.Wait Until Element Is Visible css: modal-content timeout=10s

00:00:00.019

KEYWORD SeleniumLibrary.Page Should Contain Element css: title-row h2

00:00:00.019

TEST Calendar Toggle

00:00:00.437

TEST Delete Task

00:00:00.763

Részletes futtatási napló, amely bemutatja a Selenium interakciókat a Vue komponensekkel.



10 Tesztesetek

10.1 Autentikáció és Felhasználói Folyamatok

10.1.1 Sikeres bejelentkezés

Teszteset ID	FT-AUTH-01
Funkció / Oldal	Bejelentkezés (/auth)
Módszer	Robot Framework + Selenium (URL/Route redirect ellenőrzés)
Elvárt működés / Ellenőrzés	Érvényes hitelesítő adatok megadása után a rendszer sikeresen belépteti a felhasználót és átirányítja a /profile oldalra.
Successful Login [Documentation] Test 1: Verifies redirection to /profile. [Tags] Auth Critical Login To System \${VALID_USER} \${VALID_PASS} Location Should Be \${URL}/profile	



10.1.2 Bejelentkezés hibás adatokkal

Tesztset ID	FT-AUTH-02
Funkció / Oldal	Bejelentkezés hibás adatokkal
Módszer	Robot Framework + Selenium (DOM hibaüzenet/errorMessage validáció)
Elvárt működés / Ellenőrzés	Hibás adatok megadásakor a rendszer hibaüzenetet (errorMessage) jelenít meg, és a felhasználó az /auth oldalon marad.

Failed Login

```
[Documentation]    Test 2: Verifies error message appears.
[Tags]            Auth
Execute Javascript  window.localStorage.clear()
Go To             ${URL}/auth
Input Text        css:input[type="text"]    wrong@example.com
Input Password    css:.passwordField input  wrongpassword
Click Button      css:button[type="submit"]
# Checking for common Vue error indicators
Wait Until Element Is Visible  css:.toast, .toast-error, .error-
message, .errorMessage    timeout=5s
Location Should Be    ${URL}/auth
```



10.1.3 Védett útvonalak biztonsága

Teszteset ID	FT-AUTH-03
Funkció / Oldal	Védett útvonalak biztonsága
Módszer	Robot Framework + Selenium (Middleware/Router redirect ellenőrzés)
Elvárt működés / Ellenőrzés	Kijelentkezett állapotban a /profile oldal közvetlen eléréskor a rendszer automatikusan visszairányít az /auth oldalra.
<pre>Unauthorized Access [Documentation] Test 3: Verify redirection when logged out. [Tags] Auth # Ensure session is cleared Execute Javascript window.localStorage.clear() Go To \${URL}/profile Wait Until Location Is \${URL}/auth timeout=5s Wait Until Element Is Visible css:input[type="text"] timeout=5s</pre>	

10.1.4 Kijelentkezés

Teszteset ID	FT-AUTH-04
Funkció / Oldal	Kijelentkezés és munkamenet törlése
Módszer	Robot Framework + Selenium (Munkamenet törlés és Router redirect validáció)
Elvárt működés / Ellenőrzés	A kijelentkezés gombra kattintva a rendszer törli a hitelesítést, visszairányít az /auth oldalra, és megakadályozza a visszalépést a védett útvonalakra (pl. /tasks).
<pre>Logout Flow [Documentation] Test 12: Verify session clear and redirect. [Tags] Auth Wait And Click css:.LogoutBtn Wait Until Location Is \${URL}/auth timeout=5s # Final check: Try to go back to tasks and ensure redirect Go To \${URL}/tasks Location Should Be \${URL}/auth</pre>	



10.2 Csoportok és Együttműködés

10.2.1 Új csoport felvitele

Teszt eset ID	FT-GRP-01
Funkció / Oldal	Új csoport felvitele (/groups)
Módszer	Robot Framework + Selenium (Modal és Form interakció, dinamikus listafrissítés)
Elvárt működés / Ellenőrzés	A createGroupBtn megnyomása után a modális ablakban megadott adatok alapján az új csoport bekerül a listába.

Create A Group

```
[Documentation]    Test 8: Verify group creation modal.
[Tags]            Group
Go To             ${URL}/groups
Wait And Click    css:.createGroupBtn
Wait Until Element Is Visible    css:.modalContent    timeout=5s
Input Text        id:groupTitle    New Automated Group
Input Text        id:groupDescription    This is a test group
Wait And Click    css:.createBtn
Wait Until Page Contains    New Automated Group    timeout=5s
```



10.2.2 Felhasználó kereső

Tesztet ID	FT-GRP-02
Funkció / Oldal	Felhasználó kereső (UserSearch.vue)
Módszer	Robot Framework + Selenium (Input-alapú aszinkron DOM szűrés ellenőrzése)
Elvárt működés / Ellenőrzés	A keresőmezőbe írt kulcsszó alapján a rendszer listázza a találatokat a result-item osztályú elemekben.

```
Search For Users
[Documentation]    Test 9: Verify user search filtering.
[Tags]            Group
Go To             ${URL}/groups
Wait And Click    css:.ListGroupItem
Wait Until Element Is Visible    css:.user-search-
container        timeout=5s
Input Text        css:.search-input    testuser
Wait Until Element Is Visible    css:.result-item    timeout=5s
Page Should Contain    testuser
```



10.3 Feladatkezelés

10.3.1 Feladat rögzítése

Teszt eset ID	FT-TASK-01
Funkció / Oldal	Feladat rögzítése (CreateTaskModal.vue)
Módszer	
Elvárt működés / Ellenőrzés	A cím és a dátumválasztó (CustomDatePicker) használata után a feladat mentésre kerül és megjelenik a feladatok között.

```
Create A New Task
  [Documentation]  Test 4: Open CreateTaskModal, fill form, verify
update.
  [Tags]    Task
Go To      ${URL}/tasks
Wait And Click  css:.btnNewTask
Wait Until Element Is Visible  css:.modalContent  timeout=5s
Input Text      css:.formGroup input[type="text"]  New Automated Task
Wait And Click  css:.date-input
Wait And Click  css:.day-cell.today
Click Button    css:.btnCreate
Wait For Toast  New Automated Task
```



10.3.2 Feladat részletei

Tesztset ID	FT-TASK-02
Funkció / Oldal	Feladat részletei (TaskDetailModal.vue)
Módszer	Robot Framework + Selenium (Aszinkron modális ablak adatbetöltés ellenőrzése)
Elvart működés / Ellenőrzés	Egy létező feladat "Megtekintés" gombjára kattintva a rendszer megnyitja a részletező modális ablakot, amely tartalmazza a feladat címét.
<pre>Task Detail View [Documentation] Test 5: Click a task and verify detail modal. [Tags] Task Go To \${URL}/tasks Wait Until Element Is Visible css:.fadeInItem timeout=5s Wait And Click css:.fadeInItem .viewBtn Wait Until Element Is Visible css:.modal-content timeout=10s Page Should Contain Element css:.title-row h2 Click Button css:.btnCreate Wait For Toast New Automated Task</pre>	



10.3.3 Nézetváltás

Teszt eset ID	FT-TASK-03
Funkció / Oldal	Nézetváltás (TaskCalendar.vue)
Módszer	Robot Framework + Selenium (Dinamikus komponens- láthatósági ellenőrzés)
Elvárt működés / Ellenőrzés	A naptár váltó gombra kattintva a listás nézet helyett megjelenik a naplár rács (calendar-grid) és a hozzá tartozó felületi elemek.

Calendar Toggle

```
[Documentation]    Test 6: Verify calendar component is visible.
[Tags]            Task
Go To             ${URL}/tasks
Wait Until Element Is Visible    css:.tasksArea    timeout=5s
# Attempt to click calendar button if it exists (mobile/toggle mode)
Run Keyword And Ignore Error    Click
Button    xpath://button[contains(., 'Calendar')]
Wait Until Element Is Visible    css:.calendarArea, .calendar-
grid    timeout=5s
```



10.3.4 Feladat eltávolítása

Teszt eset ID	FT-TASK-04
Funkció / Oldal	Feladat eltávolítása
Módszer	Robot Framework + Selenium (Kétlépcsős megerősítés/AlertModal validáció)
Elvárt működés / Ellenőrzés	A törlés gombra kattintva megjelenik az alert-overlay megerősítő ablak, igazolva a művelet biztonsági lépését.

Delete Task

[Documentation] Test 7: Verify delete confirmation modal.

[Tags] Task

Go To `${URL}/tasks`

Wait Until Element Is Visible `css:.fadeInItem` `timeout=5s`

Wait And Click `css:.fadeInItem .viewBtn`

Wait Until Element Is Visible `css:.modal-content` `timeout=5s`

Wait And Click `css:.btn-danger`

Wait Until Element Is Visible `css:.alert-overLay` `timeout=5s`

Page Should Contain Element `css:.alert-content`



10.4 Felhasználói Felület és Beállítások

10.4.1 Felhasználói adatok módosítása

Teszt eset ID	FT-SET-01
Funkció / Oldal	Felhasználói adatok módosítása
Módszer	Robot Framework + Selenium (Adatperzisztencia teszt és Rollback stratégia)
Elvárt működés / Ellenőrzés	Az adatok módosítása után sikerességi Toast üzenet érkezik. A teszt végén az adatok visszaállnak az eredeti állapotra a környezet stabilitása érdekében.

Update Profile

[Documentation] Test 10: Change info and verify Success Toast.

[Tags] Profile

Go To \${URL}/profile

Wait And Click css:.editBtn

Wait Until Element Is Visible css:.editInput timeout=5s

Input Text css:.editInput UpdatedName

Click Button css:.saveBtn

Wait For Toast Success

Revert the username so that subsequent test suites can still log in

Wait And Click css:.editBtn

Wait Until Element Is Visible css:.editInput timeout=5s

Input Text css:.editInput testuser1234

Click Button css:.saveBtn

Wait For Toast Success



10.4.2 Nyelvváltás

Teszteset ID	FT-SET-02
Funkció / Oldal	Nyelvváltás (i18n.js)
Módszer	Robot Framework + Selenium (Dinamikus i18n szövegcsere validáció újratöltés nélkül)
Elvárt működés / Ellenőrzés	A nyelvváltó gomb megnyomása után a felület szövegezése (pl. "Feladatok") azonnal magyarra vált.

```
Language Switcher
[Documentation]    Test 11: Verify i18n changes text to Hungarian.
[Tags]            i18n
Go To             ${URL}/tasks
Wait And Click    css:.LangBtn
Wait Until Page Contains    Feladatok    timeout=10s
```



11 Melléklet: Forráskód kivonat

11.1 *Resources.resource* (Közös változók és kulcsszavak):

```
*** Variables ***
${URL}          http://localhost:5173
${BROWSER}      Firefox
${VALID_USER}   testuser1234
${VALID_PASS}   password123

*** Keywords ***

Open Browser To Calentasker
    [Arguments]    ${path}=auth
    Open Browser    ${URL}/${path}    ${BROWSER}
    Maximize Browser Window
    Set Selenium Implicit Wait    5 seconds

Login To System
    [Arguments]    ${email}=${VALID_USER}    ${pass}=${VALID_PASS}
    Go To    ${URL}/auth
    Wait Until Element Is Visible    css:input[type="text"]    timeout=10s
    Input Text    css:input[type="text"]    ${email}
    Input Password    css:.passwordField input    ${pass}
    Click Button    css:button[type="submit"]
    Wait Until Location Is    ${URL}/profile    timeout=10s

Wait And Click
    [Arguments]    ${locator}
    Wait Until Element Is Visible    ${locator}    timeout=5s
    Click Element    ${locator}

Wait For Toast
    [Arguments]    ${text}
    # Matches common Vue toast classes
    Wait Until Page Contains    ${text}    timeout=5s
```

A tesztfájlok (*AuthTests.robot*, *TaskTests.robot*, *GroupTests.robot*, *SettingsUI.robot*) a projekt forráskódjában, a */Tests* mappában érhetőek el.



Backend

Teszt dokumentáció





12 Tesztesetek

12.1 Autentikáció és Felhasználói Folyamatok

12.1.1 Felhasználó regisztrációja

Teszteset ID	TC-USER-01
API végpont	/api/users/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A username, email, password, first_name és last_name megadása sikeresen létrehoz egy inaktív felhasználói fiókot.

```
def test_user_registration(self):
    """Test user registration process"""
    data = {
        'username': 'newuser',
        'email': 'newuser@example.com',
        'password': 'password123',
        'first_name': 'New',
        'last_name': 'User'
    }
    res = self.client.post(self.users_url, data)
    self.assertEqual(res.status_code, status.HTTP_201_CREATED)
    self.assertFalse(res.data['is_active'])
```



12.1.2 Fiók aktiválása

Teszt eset ID	TC-USER-02
API végpont	/api/users/activate/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A regisztrációs e-mailben kapott uid és token átadása sikeresen aktiválja a felhasználót.

```
def test_user_activation(self):
    """Test token activation for a newly registered user"""
    # Create an inactive user manually
    new_user = User.objects.create_user(
        username='inactive_user',
        email='inactive@example.com',
        password='password123',
        is_active=False
    )

    # Simulate clicking email link
    uid = urlsafe_base64_encode(force_bytes(new_user.pk))
    token = default_token_generator.make_token(new_user)

    activate_res = self.client.post(f"{self.users_url}activate/", {
        'uid': uid,
        'token': token
    })
    self.assertEqual(activate_res.status_code, status.HTTP_200_OK)

    new_user.refresh_from_db()
    self.assertTrue(new_user.is_active)
```




12.1.3 Felhasználó bejelentkezése

Teszt eset ID	TC-AUTH-01
API végpont	/api/login/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Érvényes hitelesítő adatok (pl. admin/admin) megadása esetén a rendszernek egy hitelesítési tokent kell visszaadnia.

```
def test_user_login(self):  
    """Test user login returns a token and user details"""  
    response = self.client.post(self.login_url, {  
        'username': 'user1',  
        'password': 'password123'  
    })  
    self.assertEqual(response.status_code, status.HTTP_200_OK)  
    self.assertIn('token', response.data)  
    self.assertEqual(response.data['username'], 'user1')
```



12.1.4 Fiók törlése

Teszt eset ID	TC-DEL-01
API végpont	/api/users/{id}
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt felhasználót soft delete eljárás alapján törli (inaktívvá állítja)

```
def test_user_soft_deletion(self):
    """Test that deleting a user marks them as inactive"""
    self.client.force_authenticate(user=self.user2)

    # We delete user2 to test soft deletion
    res = self.client.delete(f"{self.users_url}{self.user2.id}/")
    self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

    # Retrieve from DB to check soft delete
    self.user2.refresh_from_db()
    self.assertFalse(self.user2.is_active)
```



12.2 Csoportok és Együttműködés

12.2.1 Csoportok létrehozása

Teszt eset ID	TC-GRP-01
API végpont	/api/groups/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Új csoportot hoz létre a groupname és description paraméterekkel

```
def test_group_creation(self):
    """Test group creation automatically adds creator as leader"""
    self.client.force_authenticate(user=self.user1)

    # Create group
    group_data = {
        'groupname': 'Test Project Group',
        'description': 'A descriptive group'
    }
    res = self.client.post(self.groups_url, group_data)
    self.assertEqual(res.status_code, status.HTTP_201_CREATED)
    group_id = res.data['id']

    # Ensure creator is leader
    leader_member = GroupMember.objects.get(group_id=group_id,
user=self.user1)
    self.assertEqual(leader_member.role, 'leader')
```



12.2.2 Csoporttag hozzáadása

Teszt eset ID	TC-GRP-02
API végpont	/api/group-members/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Sikeresen hozzárendel egy felhasználót (user) egy csoporthoz (group) a megadott szerepkörrel (pl. „reader”).

```
def test_member_addition(self):
    """Test adding another member to an existing group"""
    self.client.force_authenticate(user=self.user1)

    group = Group.objects.create(
        groupname="Test Add Member Group",
        created_by_userid=self.user1
    )
    GroupMember.objects.create(group=group, user=self.user1,
role='leader')

    # Add user2 to the group as an operator
    member_data = {
        'group': group.id,
        'user': self.user2.id,
        'role': 'operator'
    }
    add_res = self.client.post(self.group_members_url, member_data)
    self.assertEqual(add_res.status_code, status.HTTP_201_CREATED)

    member_count = GroupMember.objects.filter(group_id=group.id).count()
    self.assertEqual(member_count, 2)
```



12.2.3 Csoport törlése

Teszt eset ID	TC-GRP-03
API végpont	/api/groups/{id}
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt csoportot soft delete eljárás alapján törli (inaktívvá állítja)

```
def test_group_soft_deletion(self):
    """Test that deleting a group marks it as inactive"""
    self.client.force_authenticate(user=self.user1)

    group = Group.objects.create(
        groupname="Group to delete",
        created_by_userid=self.user1
    )
    GroupMember.objects.create(group=group, user=self.user1,
role='leader')

    res = self.client.delete(f"{self.groups_url}{group.id}/")
    self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

    # Retrieve from DB to check soft delete
    group.refresh_from_db()
    self.assertFalse(group.active)
```



12.3 Feladatkezelés

12.3.1 Személyes feladat hozzáadása

Teszt eset ID	TC-TASK-01
API végpont	/api/tasks/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A rendszer létrehoz egy feladatot, ha a title, description, priority, status, created_by_userid és assigned_to_userid

```
def test_personal_task_creation(self):
    """Test creating a personal task"""
    self.client.force_authenticate(user=self.user1)

    personal_task_data = {
        'title': 'Buy groceries',
        'description': 'Milk, Eggs, Bread',
        'priority': 'medium',
        'status': 'todo',
        'created_by_userid': self.user1.id,
        'assigned_to_userid': self.user1.id,
    }
    p_res = self.client.post(self.tasks_url, personal_task_data)
    self.assertEqual(p_res.status_code, status.HTTP_201_CREATED)
    self.assertIsNone(p_res.data.get('group_detail'))
```



12.3.2 Csoportos feladat létrehozása

Teszt eset ID	TC-TASK-02
API végpont	/api/tasks/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Sikeresen létrehoz egy adott csoporthoz rendelt feladatot, ha a group azonosító szerepel a kérés törzsében.

```
def test_group_task_creation(self):
    """Test creating a group task"""
    self.client.force_authenticate(user=self.user1)

    # Group Task setup (Creating group first)
    group = Group.objects.create(groupname='Work', description='Work
tasks', created_by_userid=self.user1)
    GroupMember.objects.create(group=group, user=self.user1,
role='leader')
    GroupMember.objects.create(group=group, user=self.user2,
role='operator')

    # Create Task for Group
    group_task_data = {
        'title': 'Finish Backend Tests',
        'description': 'Write comprehensive unit tests',
        'priority': 'high',
        'status': 'todo',
        'group': group.id,
        'created_by_userid': self.user1.id,
        'assigned_to_userid': self.user1.id,
    }
    g_res = self.client.post(self.tasks_url, group_task_data)
    self.assertEqual(g_res.status_code, status.HTTP_201_CREATED)

    # Verify assignment
    task = Task.objects.get(id=g_res.data['id'])
    self.assertEqual(task.assigned_to_userid.id, self.user1.id)
```



12.3.3 Feladat módosítása

Teszt eset ID	TC-TASK-03
API végpont	/api/tasks/{id}/
HTTP metódus	PATCH
Elvárt működés / Ellenőrzés	Frissített mezők küldése (pl. {„title”: „Frissített cím”}) módosítja a feladatot, és a változtatás mentésre kerül az adatbázisban.

```
def test_task_edit(self):
    """Test editing a task's properties"""
    self.client.force_authenticate(user=self.user1)

    # Create a task to edit
    task = Task.objects.create(
        title='Initial Title',
        description='Initial Description',
        priority='medium',
        status='todo',
        created_by_userid=self.user1,
    )

    edit_data = {
        'title': 'Updated Title',
        'status': 'in_progress'
    }
    res = self.client.patch(f"{self.tasks_url}{task.id}/", edit_data)

    self.assertEqual(res.status_code, status.HTTP_200_OK)

    task.refresh_from_db()
    self.assertEqual(task.title, 'Updated Title')
    self.assertEqual(task.status, 'in_progress')
    self.assertEqual(task.description, 'Initial Description')
```




12.3.4 Feladatok szűrése

Teszt eset ID	TC-TASK-05
API végpont	/api/tasks/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	A lekérdezési paraméterek használata sikeresen kiszűri és visszaadja a megadott paramétereknek megfelelő feladatokat. Ez támogatja a Frontend szűrési követelményeit.

```
def test_filter_tasks(self):
    """Test filtering tasks by status, group, and group_isnull"""
    self.client.force_authenticate(user=self.user1)

    # Setup specific groups and tasks
    group1 = Group.objects.create(groupname="Filter Group",
    created_by_userid=self.user1)
    GroupMember.objects.create(group=group1, user=self.user1,
    role='leader')

    task_todo_personal = Task.objects.create(title="T1",
    created_by_userid=self.user1, status='todo')
    task_inprog_personal = Task.objects.create(title="T2",
    created_by_userid=self.user1, status='in_progress')
    task_done_group = Task.objects.create(title="T3",
    created_by_userid=self.user1, status='done', group=group1)

    # Filter by status
    res_status = self.client.get(f"{self.tasks_url}?status=todo")
    self.assertEqual(res_status.status_code, status.HTTP_200_OK)
    ids = [t['id'] for t in res_status.data]
    self.assertIn(task_todo_personal.id, ids)
    self.assertNotIn(task_inprog_personal.id, ids)

    # Filter by group
    res_group = self.client.get(f"{self.tasks_url}?group={group1.id}")
    self.assertEqual(res_group.status_code, status.HTTP_200_OK)
    # Verify only task_done_group is in the result (among the ones we
    created)
    group_ids = [t['id'] for t in res_group.data]
    self.assertIn(task_done_group.id, group_ids)
    self.assertNotIn(task_todo_personal.id, group_ids)

    # Filter by group IS NULL (Personal Tasks)
    res_null = self.client.get(f"{self.tasks_url}?group_isnull=true")
```



```
self.assertEqual(res_null.status_code, status.HTTP_200_OK)
null_ids = [t['id'] for t in res_null.data]
self.assertIn(task_todo_personal.id, null_ids)
self.assertNotIn(task_done_group.id, null_ids)
```

12.3.5 Feladat törlése

Teszt eset ID	TC-TASK-04
API végpont	/api/tasks/{id}/
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	A feladatot inaktívvá teszi

```
def test_task_soft_deletion(self):
    """Test that deleting a task marks it as inactive instead of
    removing it from DB"""
    self.client.force_authenticate(user=self.user1)

    task = Task.objects.create(
        title="Task to delete",
        created_by_userid=self.user1,
        priority='high',
        status='todo'
    )

    res = self.client.delete(f"{self.tasks_url}{task.id}/")
    self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

    # Retrieve from DB to check soft delete
    task.refresh_from_db()
    self.assertFalse(task.active)
```



12.4 Lekérések

12.4.1 Felhasználók lekérése

Teszt eset ID	TC-GET-01
API végpont	/api/users/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden felhasználót visszaad
<pre>def test_list_users(self): """Test retrieving all active users""" self.client.force_authenticate(user=self.user1) res_list = self.client.get(self.users_url) self.assertEqual(res_list.status_code, status.HTTP_200_OK) self.assertGreaterEqual(len(res_list.data), 2)</pre>	

12.4.2 Adott felhasználó lekérése

Teszt eset ID	TC-GET-02
API végpont	/api/users/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt felhasználót visszaadja
<pre>def test_retrieve_user(self): """Test retrieving a specific user by ID""" self.client.force_authenticate(user=self.user1) res_detail = self.client.get(f"{self.users_url}{self.user2.id}/") self.assertEqual(res_detail.status_code, status.HTTP_200_OK) self.assertEqual(res_detail.data['id'], self.user2.id) self.assertEqual(res_detail.data['username'], self.user2.username)</pre>	



12.4.3 Csoportok lekérése

Teszteteset ID	TC-GET-03
API végpont	/api/groups/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden csoportot visszaad

```
def test_list_groups(self):
    """Test retrieving all groups"""
    self.client.force_authenticate(user=self.user1)
    Group.objects.create(groupname="Global Group 1",
created_by_userid=self.user1)
    Group.objects.create(groupname="Global Group 2",
created_by_userid=self.user1)

    res_list = self.client.get(self.groups_url)
    self.assertEqual(res_list.status_code, status.HTTP_200_OK)
    self.assertGreaterEqual(len(res_list.data), 2)
```

12.4.4 Adott csoport lekérése

Teszteteset ID	TC-GET-04
API végpont	/api/groups/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt csoportot visszaadja

```
def test_retrieve_group(self):
    """Test retrieving a specific group by ID"""
    self.client.force_authenticate(user=self.user1)
    group1 = Group.objects.create(groupname="Specific Group 1",
created_by_userid=self.user1)

    res_detail = self.client.get(f"{self.groups_url}{group1.id}/")
    self.assertEqual(res_detail.status_code, status.HTTP_200_OK)
    self.assertEqual(res_detail.data['id'], group1.id)
    self.assertEqual(res_detail.data['groupname'], group1.groupname)
```



12.4.5 Feladatok lekérése

Teszt eset ID	TC-GET-05
API végpont	/api/tasks/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden feladatot visszaad

```
def test_list_tasks(self):
    """Test retrieving all tasks"""
    self.client.force_authenticate(user=self.user1)
    Task.objects.create(title="Task A", created_by_userid=self.user1,
status='todo')
    Task.objects.create(title="Task B", created_by_userid=self.user1,
status='in_progress')

    res_list = self.client.get(self.tasks_url)
    self.assertEqual(res_list.status_code, status.HTTP_200_OK)
```

12.4.6 Adott feladat lekérése

Teszt eset ID	TC-GET-06
API végpont	/api/tasks/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt feladatot visszaadja

```
def test_retrieve_task(self):
    """Test retrieving a specific task by ID"""
    self.client.force_authenticate(user=self.user1)
    task1 = Task.objects.create(title="Task A",
created_by_userid=self.user1, status='todo')

    res_detail = self.client.get(f"{self.tasks_url}{task1.id}/")
    self.assertEqual(res_detail.status_code, status.HTTP_200_OK)
    self.assertEqual(res_detail.data['id'], task1.id)
    self.assertEqual(res_detail.data['title'], task1.title)
```